

# Reinforcement Learning pentru Conducere Autonomă într-un Environment Personalizat "Highway"

Echipa Juno  
Corobana Ingrid  
Moise Irina  
Prizlopan Iustin  
Sescu Matei

Ianuarie 2026

## 1 Formularea problemei

Obiectivul proiectului este antrenarea unui agent de control pentru un vehicul ego (masina galbena) într-un mediu de trafic. Agentul trebuie sa circule pe un drum cu mai multe benzi, sa evite coliziunile cu vehiculele din trafic (masinile albastre), sa ramana pe carosabil si, in acelasi timp, sa mentina o viteza rezonabila. Problema este formulata ca o problema de Reinforcement Learning in care agentul invata o politica de control doar pe baza recompensei primite din mediu.

### 1.1 Stare, actiune, recompensa

Modelam problema in termeni de *stare*, *actiune* si *recompensa* astfel:

- **Stare:** observatia de baza a mediului este de tip `"Kinematics"` din `exttthighway_env`. Dupa aplicarea lui `FlatObsWrapper`, obtinem un vector continuu de dimensiune 25, care contine pozitia, viteza si orientarea vehiculului ego, precum si informatii agregate despre cateva vehicule din jur.
- **Actiune:** mediul original este cu actiuni continue (acceleratie, directie). Pentru agentii discreti folosim un wrapper `DiscreteActionWrapper` care defineste 5 actiuni: *Idle* (0), *Accelerate* (1), *Brake* (2), *Left* (3), *Right* (4).
- **Recompensa:** functia de recompensa din `custom_env.py` combina mai multe componente: penalizari puternice pentru coliziune sau iesirea de pe drum si termeni pozitivi pentru viteza ridicata aliniata cu sensul benzii si pentru positionarea aproape de centrul benzii (lane centering). In acest fel, agentul primeste feedback la fiecare pas de timp, nu doar la finalul episodului.

Diagrama generala agent-mediu este prezentata in Figura 1.

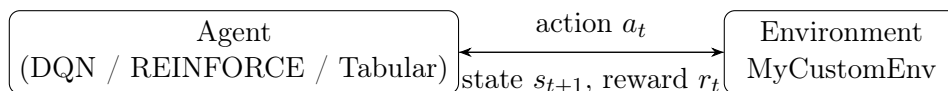


Figure 1: Schema generala agent-mediu folosita in proiect.

## 2 Environment Design

Mediul este construit plecand de la pachetul `highway_env`, dar logica drumului si a traficului a fost rescrisa in fisierul `custom_env.py`. Drumul este reprezentat ca un graf de benzi (`RoadNetwork`) cu mai multe sectiuni: un segment drept de intrare, o zona de imbinare (`merge`) cu o banda laterala, un viraj circular, portiuni drepte suplimentare si o iesire verticala catre un nod final `c0`. Sunt definite mai multe benzi folosind clasele `exttttStraightLane` si `CircularLane`, inclusiv o banda de intrare laterala pentru vehiculele care se incadreaza in trafic.

Vehiculele din trafic sunt de tip `IDMVehicle` (model de conducere IDM) si sunt plasate atat pe sectiuni fixe (obstacole pozitionate manual), cat si aleator pe drum. Pentru unele vehicule folosim metoda `plan_route_to` pentru a le forta sa urmeze un traseu prin retea (de exemplu, de la nodul `b0` la `c0`). La fiecare pas de timp, `extttthighway_env` actualizeaza automat dinamica tuturor vehiculelor.

Episodul se termina atunci cand vehiculul ego se ciocneste sau paraseste drumul, iar trunchierea se produce cand timpul depaseste `config["duration"]`. Aceste conditii asigura episoade finite si un semnal de invatare bine definit pentru toti agentii.

## 3 Algoritmi

In acelasi mediu am antrenat si comparat trei tipuri de agenti: un agent tabular (Q-Learning), un agent DQN si un agent de tip policy gradient (REINFORCE cu baseline de valoare).

### 3.1 Tabular Q-Learning

Agentul tabular foloseste o reprezentare discretizata a starii, obtinuta prin wrapper-ul `extttTabularObsWrapper`. Observatia continua este proiectata intr-un vector de 6 componente: pozitia ego pe o grila 2D ( $x, y$ ), un nivel discretizat de combustibil si doua variabile discrete care codifica directia aproximativa catre tinta ( $dx, dy$ ), impreuna cu o componenta binara simpla de context. Dimensiunile de discretizare sunt  $[6, 6, 3, 2, 3, 3]$ , rezultand 1944 de stari posibile.

Pentru fiecare stare discreta se pastreaza o valoare  $Q(s, a)$  intr-un tabel cu 5 actiuni. Actualizarea este cea clasica de Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)),$$

cu factor de invatare  $\alpha = 0,05$ , factor de discount  $\gamma = 0,99$  si o strategie epsilon-greedy pentru explorare, cu  $\epsilon$  care scade treptat de la 1 la 0.05.

### 3.2 Deep Q-Network (DQN)

Agentul DQN foloseste direct vectorul continuu de 25 de dimensiuni ca intrare intr-o retea neuronală feed-forward. Arhitectura implementata in proiect este:

- strat de intrare: vectorul de stare (25 dimensiuni);
- doua straturi ascunse dense cu 256 si respectiv 128 de neuroni si activare ReLU;
- strat de iesire: 5 neuroni care produc valorile  $Q(s, a)$  pentru fiecare actiune discreta.

Reteaua este antrenata folosind experience replay (buffer de 50 000 de tranzitii) si un *target network* sincronizat periodic. La fiecare pas, o tranzitie  $(s, a, r, s', done)$  este adaugata in buffer, iar cand exista suficiente exemple se esantioneaza mini-loturi de marime 64. Tintelor de invatare li se aplica fie formula clasica DQN, fie varianta Double DQN (selectie de actiune cu `policy_net`, evaluare cu `target_net`). Functia de pierdere este Huber (`SmoothL1Loss`). Explorarea se face cu o strategie epsilon-greedy cu  $\epsilon$  care scade de la 1.0 la 0.01.

Diagrama de arhitectura DQN este prezentata in Figura 2.

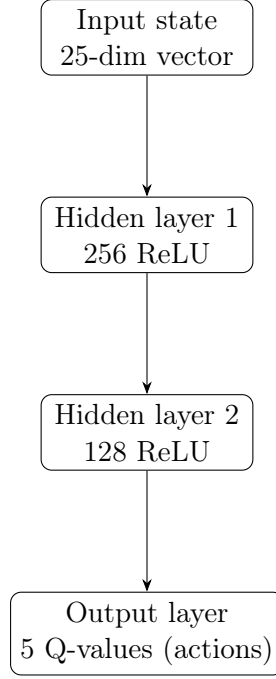


Figure 2: Arhitectura DQN care mapeaza starea la valorile  $Q(s, a)$  pentru cele 5 actiuni.

### 3.3 Policy Gradient (REINFORCE + Baseline)

Al treilea agent este unul de tip policy-based. Politica  $\pi_\theta(a \mid s)$  este parametrizata de o retea neuronală (PolicyNetwork) cu un singur strat ascuns de 64 de neuroni si iesire softmax peste cele 5 actiuni. In plus, am introdus un *value network* care aproximeaza functia de valoare  $V_\phi(s)$  si este antrenat impreuna cu politica.

Update-ul foloseste REINFORCE cu advantage:

$$\nabla J(\theta) \approx \sum_t \log \pi_\theta(a_t \mid s_t) \cdot A_t, \quad A_t = G_t - V_\phi(s_t),$$

unde  $G_t$  este returnul Monte Carlo din pasul  $t$ . Valoarea este antrenata cu eroare patratica fata de  $G_t$ , iar pierderea totala este suma dintre pierderea de politica si o componenta de valoare ponderata. Acest baseline reduce varianta gradientului si imbunatateste stabilitatea fata de REINFORCE simplu.

## 4 Experimente

Obiectivul experimental a fost compararea celor trei agenti (tabular, DQN si REINFORCE cu baseline) in exact acelasi mediu si cu acelasi set de actiuni discrete. Pentru fiecare agent am definit un script separat de antrenare ('train\_tabular.py', 'train\_dqn.py', 'train\_reinforce.py') si am pastrat aceleasi conditii de initializare ale mediului.

Pentru DQN si agentul tabular am rulat cate 500 de episoade de antrenare, in timp ce pentru REINFORCE am folosit 1000 de episoade pentru a compensa varianta mai mare a gradientului de politica. La fiecare episod am inregistrat recompensa cumulata si am salvat un model „best” pe baza mediei mobile a ultimelor 20–50 de episoade.

Pentru a vizualiza evolutia antrenarii, scriptul 'plot\_results.py' incarca fisierele CSV generate de fiecare agent si deseneaza curbele de recompensa, impreuna cu o versiune netezita. Figura de mai jos ('training\_comparison.png') arata clar viteza de convergenta si nivelul final de performanta pentru fiecare algoritm.

## 4.1 Curbe de antrenare

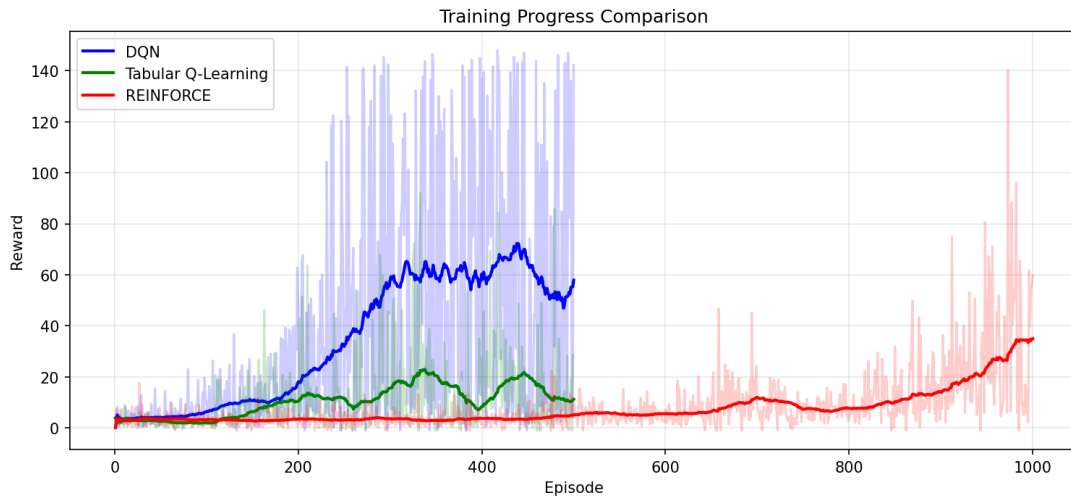


Figure 3: Curbele de recompensa in timpul antrenarii pentru toti cei trei agenti.

## 4.2 Evaluare si comportament

In plus, am implementat un script de evaluare ‘evaluate\_agents.py’ care incarca modelele antrenate si ruleaza un numar fix de episoade de test pentru fiecare agent, cu politica pur determinista (fara explorare,  $\epsilon = 0$ ). Rezultatele sunt salvate in fisiere CSV si reprezentate grafic intr-un barplot (‘eval\_barplot.png’), care permite compararea directa a recompenselor medii.

Comportamentul calitativ al agentilor a fost analizat si prin scriptul ‘visualize\_actions.py’, care deseneaza pe harta pozitiile masinii ego, colorate in functie de actiunea aleasa (accelerare, franare, schimbare de banda etc.). Aceste harti de actiuni ajuta la intelegerea tiparelor invatate de fiecare algoritm.

## 5 Analiza si discutii

Rezultatele numerice arata ca agentul DQN obtine in medie cele mai mari recompense, atat pe parcursul antrenarii, cat si in episoadele de evaluare. Curba sa de invatare creste relativ rapid si se stabilizeaza la un nivel inalt, semn ca reseaua a invatat o politica robusta de mentinere a benzii, evitarea coliziunilor si control al vitezei.

Agentul tabular Q-Learning reuseste sa invete un comportament rezonabil, dar ramane semnificativ mai slab decat DQN. Desi discretizarea de 6 dimensiuni este gestionabila ca marime a tabelului, ea pierde informatii fine despre dinamica vehiculelor din jur, iar actualizarea prin esantionare dintr-un mediu complex ramane dificila. Totusi, comparativ cu un agent aleator, tabularul imbunatateste clar frecventa coliziunilor si calitatea traiectoriei.

Agentul REINFORCE cu baseline obtine performante intermediare: este mai bun decat tabularul, dar in medie mai slab decat DQN. Introducerea unei retele de valoare a redus varianta si a stabilitat invatarea fata de o versiune fara baseline, dar metoda ramane sensibila la hiperparametri (rata de invatare, normalizarea avantajelor, lungimea episoadelor). Faptul ca foloseste doar returnuri Monte Carlo, fara bootstrapping, face ca invatarea sa fie mai lenta si mai zgomo-toasa.

Vizualizarile actiunilor confirma aceste concluzii: DQN tinde sa pastreze o traiectorie mai fluida, cu schimbari de banda planificate si viteza constanta, in timp ce REINFORCE afiseaza uneori oscilatii intre accelerare si franare. Agentul tabular ia decizii mai „rigide”, fiind limitat de discretizarea mai grosiera a starii.

## 6 Concluzii si directii viitoare

In acest proiect am definit un mediu personalizat de conducere pe baza `extttthighway_env` si am comparat trei abordari de invatare prin intarire: Q-Learning tabular, Deep Q-Network si REINFORCE cu baseline. Rezultatele experimentale arata ca, pentru o reprezentare de stare continua si relativ bogata (25 de dimensiuni), metodele bazate pe functii de aproximare profunde (DQN) ofera un avantaj clar fata de un tabel discret simplu. In acelasi timp, un agent policy-based cu baseline poate atinge performante rezonabile, dar necesita o reglare mai atenta a hiperparametrilor.

Ca directii viitoare, proiectul poate fi extins in mai multe moduri:

- trecerea la algoritmi actor-critic mai avansati (PPO, A2C), care combina avantajele bootstrapping-ului cu stabilitatea unui objective cu constrangeri;
- imbunatatirea functiei de recompensa si a configuratiei de trafic, pentru a viza scenarii mai realiste (depasiri, intersectii, limitari de viteza variabile);
- analiza robustetii politicilor invatate la variatii ale dinamici vehiculelor sau ale senzorilor (de exemplu zgomot pe observatii);
- integrarea proiectului intr-o interfata grafica interactiva pentru demonstratii in timp real.

Per ansamblu, proiectul demonstreaza modul in care conceptele teoretice de RL (stare, actiune, recompensa, Q-Learning, DQN, policy gradient) pot fi aplicate intr-un scenariu de conducere autonoma simplificat, respectand cerintele de implementare, experimentare si documentare ale cursului.